

Hardware In The Loop For Grid Resilience

Final Design Report

Team 5: Alec Benedict, Janessa Green, Kendall Meienhofer, Emily Ninestein

University of Vermont

April 29, 2025

Date	Revision	Release Notes
4/25/2025	01	Document created
4/29/2025	02	Document finalized for submission

Contents

1	Executive Summary	3
2	Introduction	3
3	Problem Definition	3
4	Design Description	6
4.1	System Overview	6
4.2	Design Elements	6
4.2.1	ePHASORSIM and PSS/E	6
4.2.2	RT-LAB and OPAL-RT	11
4.2.3	Inverter Controller	12
4.3	Alternative Design Considerations	12
4.3.1	Inverter to Arduino	12
5	Design Evaluation	13
5.1	Technical Analysis	13
5.1.1	Modeling the Grid in RT-LAB	13
5.1.2	Choosing a Step Size	15
5.1.3	Using an Arduino Mega Microcontroller	16
5.1.4	Scaling Signals for Arduino	16
5.2	Engineering Specification Evaluation	19
5.3	Budget Analysis	20
6	Project Reflections	21
6.1	Retrospective	21
6.2	Review	22
6.3	Lessons Learned	23

1 Executive Summary

As Vermont strives to meet its renewable energy goals, distributed energy resources (DERs) will become increasingly present on the state’s electric grid, requiring changes to the transmission system to ensure that power remains safe and reliable for all Vermonters. To support this transition, VELCO has partnered with UVM SEED to create a hardware-in-the-loop (HIL) simulation of the Vermont grid, designed to test how inverters — which connect DERs to the grid — respond to contingencies under high-DER conditions. This simulation will help VELCO maintain the reliability of the transmission system as renewable energy continues to grow.

The solution developed by the SEED team integrates a network model, using .raw and .dyr files (standard data formats used by VELCO transmission planners), into an HIL simulation that incorporates a physical inverter controller programmed with PQ control. The purpose of this report is to review the SEED team’s HIL design, provide insights and feedback to future teams working on similar HIL projects (including on using Open Modelica to improve the accuracy of dynamic device behavior), and leave the door open for continued development in this area.

2 Introduction

The purpose of this report is to present the design developed by the SEED team and explain why it successfully meets the needs of the client, VELCO. This report serves as a record of the team’s key decisions throughout the project and how those choices contributed to an effective final design. Along the way, the team encountered several pivotal moments that required a re-evaluation of the project scope or an analysis of alternative design decisions, and this report will explain the reasoning behind the choices the team ultimately made.

VELCO is seeking an HIL design to simulate the effects of DERs on the Vermont electric grid, with the goal of optimizing resilience and grid response as part of their mission to provide safe and reliable power to all Vermonters. The report will first outline the problem the HIL simulation attempts to address, then provide a detailed description of the team’s final design and its key features. Subsequently, the design will be evaluated and a justification of the important decisions the team made will be provided. Finally, reflections on the project will be presented, including a team retrospective, a review of the project, and lessons learned along the way.

3 Problem Definition

VELCO is responsible for the transmission of safe, reliable power to Vermonters. As Vermont obtains an increasing amount of its electricity from renewable sources (with a plan to meet 25% of energy needs from renewable sources by 2025, 45% by 2035, and 90% by 2050 [1]), VELCO’s transmission planners need to anticipate the impact of this renewable energy on the system. One such impact is illustrated in Figure 1, which depicts the load on the New England system on the spring daytime minimum (the date on which the midday load was at its lowest for the year) in 2021, 2022, and 2023. This figure is often referred to as “the duck curve,” and its shape is the result of increased solar photovoltaics (PV) on the system. As the sun rises in the sky throughout the day, the output of solar PV increases, and because rooftop PV is connected on the load side, this energy appears as a “negative” load, causing the dip in the load during daylight hours – the “belly” of the duck. As shown in the figure, the minimum load value has gotten lower each year. According to load forecasts, this value will be negative by the year 2033, meaning that the load side of the system will actually be pushing power back onto the system during the spring daytime minimum.

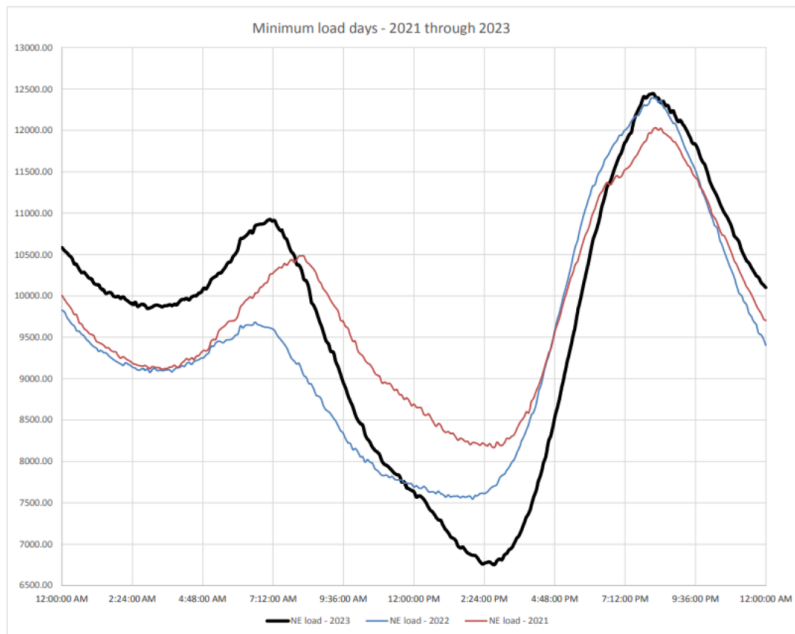


Figure 1: Spring Daytime Minimum Load Trend [2]

While this increase in solar PV is exciting, it introduces new vulnerabilities to the system that have not existed before. Historically, large power plant generators have supplied electricity to the grid at a fixed voltage and frequency, keeping the system stable. However, during the spring daytime minimum, if power is supplied entirely by distributed energy resources (DERs), the large generators will not be supplying electricity and therefore will not stabilize the voltage and frequency of the system. Rather, power will be supplied to the system through thousands of small inverters, which transform the DC power produced by the solar panels into AC power for the grid. Thus, the response of such inverters to contingency events, especially during the spring daytime minimum, is of particular interest to VELCO as they plan a resilient grid for the future.

Thus, a problem VELCO faces is that they have limited means to determine how inverters will respond to contingency events. Traditionally, transmission planners like VELCO must rely on digital simulations or data from system faults to understand how inverters will behave under stressed conditions. These existing methods have several limitations; entrusted to provide reliable transmission service, VELCO is seeking new ways to better understand the interaction between transmission stability and inverter performance.

A hardware-in-the-loop (HIL) simulation would solve this problem for VELCO, enabling them to determine the response of a physical inverter to a contingency event on their system. An HIL simulation is a combination of software and hardware that interface through digital and analog input/output (I/O) signals, allowing a physical device to be tested with a digital model. The basis of HIL simulation is a real-time simulation, meaning that the simulation is executed on the same time scale as the event being modeled – this is critical for accurately capturing the behavior of the physical device, as its response to a signal that occurs for a microsecond might be different than its response to the same signal occurring over a full second.

Such a solution would also be of value to the NEST-VT research group, which has an OPAL-RT device that enables real-time simulation but has not used it for HIL simulation yet. Thus, the proposed solution is an HIL simulation setup that uses VELCO’s system during the 2033 Spring Daytime Minimum as the grid model, NEST-VT’s OPAL-RT for the real-time simulation, and an inverter obtained by NEST-VT from the University of Texas as the device under test. This would benefit VELCO as a demonstration of how an HIL testbench could be useful for their team onsite, NEST-VT as a launch point for further HIL simulations, and distribution utilities and Vermont residents as stakeholders in the reliability of Vermont’s electric grid.

The requirements for this solution as prioritized by VELCO are included in Table 1, which includes VELCO’s ranking of importance as either “Low,” “Medium,” or “High,” and the category of each require-

ment.

Rank	Category	Requirement
High	Functional	Physically interfaces with dynamic devices (e.g., inverters).
High	Functional	Performs a powerflow (static) simulation with HIL.
High	Functional	Simulates in real-time.
High	Functional	Characterizes inverter response to the Daytime Minimum Load, High DER grid condition.
High	Functional	Reports results in an easy-to-interpret format.
High	Functional	Can be replicated by VELCO engineers on-site.
High	Functional	Includes documentation explaining how the simulation can be performed.
Medium	Functional	Performs a transient stability simulation with HIL.
Medium	Functional	Includes an integration and testing plan for PSS/E.
Medium	Functional	Includes an integration and testing plan for OPAL-RT.
Medium	Functional	Includes an integration and testing plan for an inverter.
Medium	Analytical	Characterizes the inverter's response behavior to multiple grid conditions.
Medium	Environmental	Simulation is energy efficient.
Low	Functional	Can be adapted for other HIL simulation uses.
Low	Cost	Does not require additional software or hardware to be purchased.

Table 1: Customer Requirements

4 Design Description

4.1 System Overview

To meet the client's needs, an HIL simulation was configured and performed. Figure 2 shows a high-level diagram of how the simulation setup fits together. This diagram consists of three main blocks: the host PC, OPAL-RT, and physical inverter controller, all of which are discussed in further detail below. Some elements, such as the use of Power Systems Simulator for Engineering (PSS/E) and the OPAL-RT device, were prescribed by the client, however, others required engineering analysis to select.

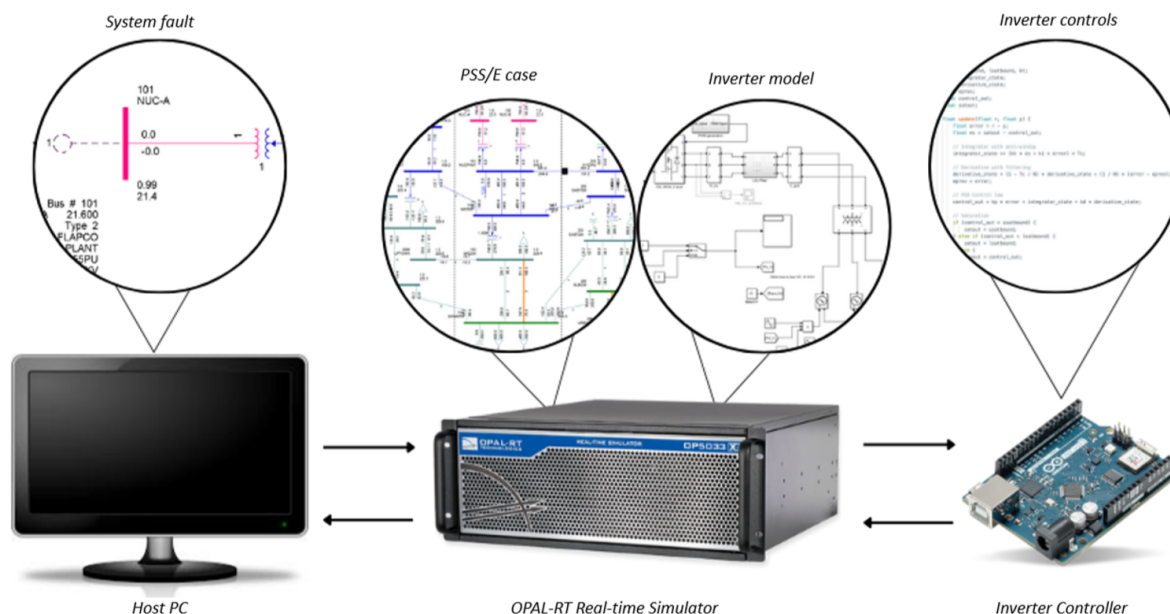


Figure 2: HIL System Overview

To perform the HIL simulation, the three elements shown in Figure 2 are physically connected, and information is passed between them. The simulation is set up on the host PC. This includes importing data files from PSS/E into RT-LAB, along with the inverter model built in Simulink. These files run on the OPAL-RT simulator, where powerflow calculations are performed. While the simulation is running, faults can be triggered directly from the host PC. Throughout the simulation, data is continuously exchanged between the OPAL-RT and the Arduino microcontroller, which acts as the inverter controller. The Arduino processes the data using its internal code that contains the PQ inverter logic and sends it back to OPAL-RT. Values can be viewed through scopes in RT-LAB on the host PC.

4.2 Design Elements

4.2.1 ePHASORSIM and PSS/E

PSS/E is a powerflow simulation software for modeling and analyzing the transmission grid. It is the software used by transmission planning engineers at VELCO when they perform studies on their system, which is why its use was prioritized in this design. PSS/E is capable of performing several different types of simulations, including steady-state powerflow, contingency analysis, and dynamic stability analysis. This project focuses on dynamic stability analysis. Dynamic stability analysis, also referred to as transient stability analysis, evaluates how the power system responds to disturbances over time. These disturbances can include faults, sudden changes in load, or the loss of a generator or transmission line. By simulating the system's dynamic behavior, transmission planning engineers can assess whether corrective actions or design changes are needed to ensure the grid can recover smoothly from unexpected events.

To incorporate a PSS/E case into the HIL simulation, dynamic stability analysis was first run on the case in PSS/E to 1) confirm that the network could successfully converge at every step in the dynamic stability simulation and 2) to establish a baseline to compare results against. For example, Figure 4 shows the dynamic stability results for an example case, 'savnw' (shown in Figure 3) with no fault, and Figure 5 shows the results for 'savnw' with a bus-to-ground fault tripped first on bus 101, then on bus 102, then on bus 3008. Each time a fault is tripped, the system was allowed to stabilize before the next fault is tripped.

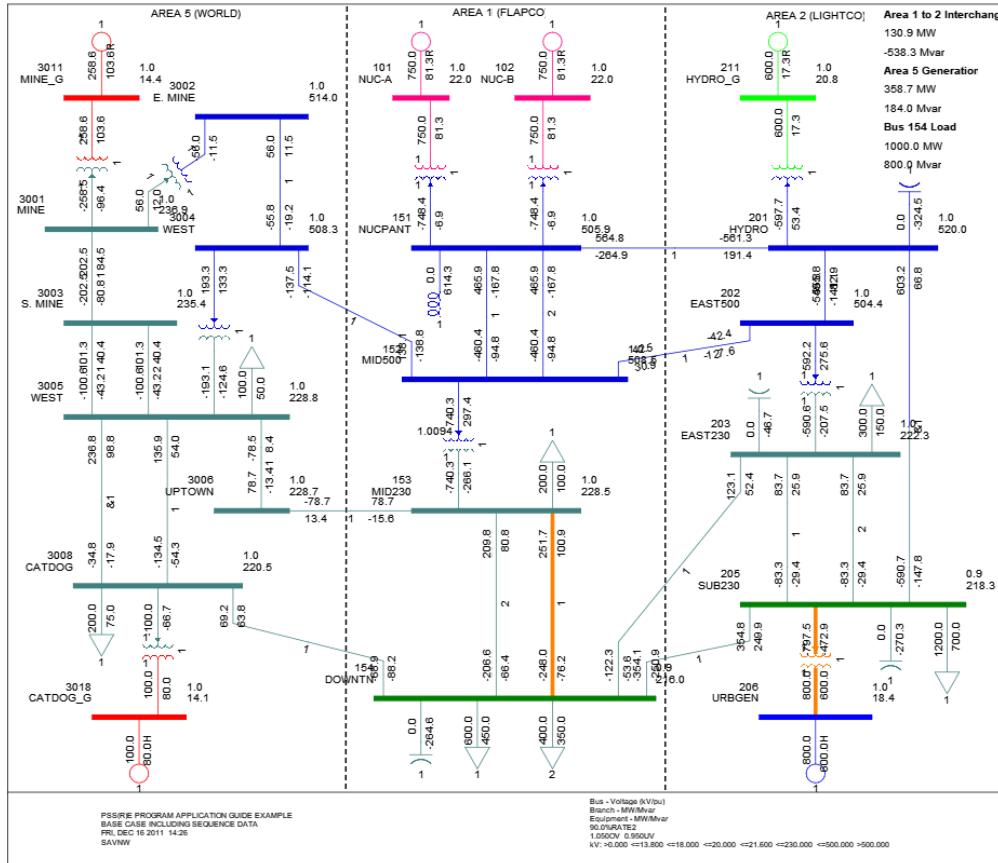


Figure 3: Example Case 'savnw' in PSS/E

Then, once it is confirmed that dynamic stability can run properly on the case in PSS/E, the case is ready to be incorporated into the HIL simulation. To do this, the case data (.raw file) and dynamic data (.dyr file) from PSS/E must be saved, uploaded to the host PC, and included in the project folder. The .raw file must be saved as version 32. Then, within the project in RT-LAB, an ePHASORSIM block must be created, as shown in Figure 6. Within the parameter popup (Figure 7), the .raw and .dyr files must be selected. Additionally, an excel sheet specifying the desired inputs and outputs to the ePHASORSIM block must be selected. An example of such an Excel file is shown in Figure 8. The appropriate syntax for this file is provided in redOPAL-RT's I/O Guide.

The ePHASORSIM block performs a very similar function as PSS/E dynamic stability analysis, but it does so in real time. One note is to be careful with how loads are modeled. If loads are 'converted' from constant MVA to constant current/impedance in PSS/E before dynamic stability analysis is performed, the case must be saved *with the loads converted* before it is uploaded to ePHASORSIM to see matching results. The ePHASORSIM solver will run the dynamic stability analysis with the loads modeled however they were modeled in the .raw file (it looks for non-zero columns to determine this).

The results of running the simulation in ePHASORSIM with no fault and with the same bus-to-ground fault on busses 101, 102, and 3008 are shown in Figures 9 and 10, respectively.

As shown in the figures, the results of dynamic stability analysis in PSS/E and ePHASORSIM align,

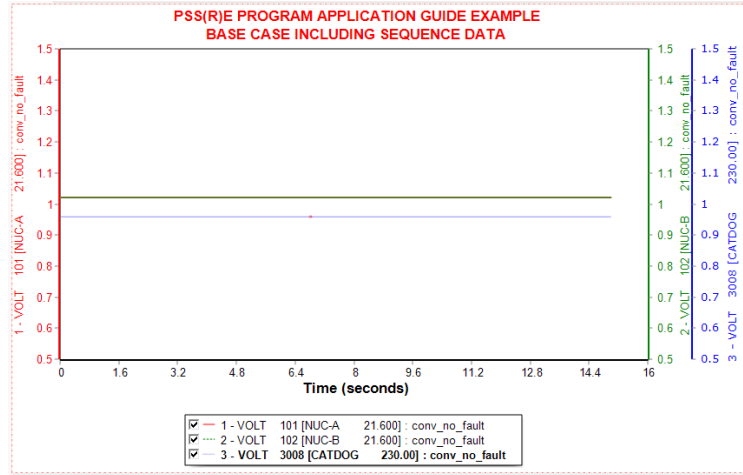


Figure 4: PSS/E Dynamic Results for Bus 101, 102, 3008 with No Fault

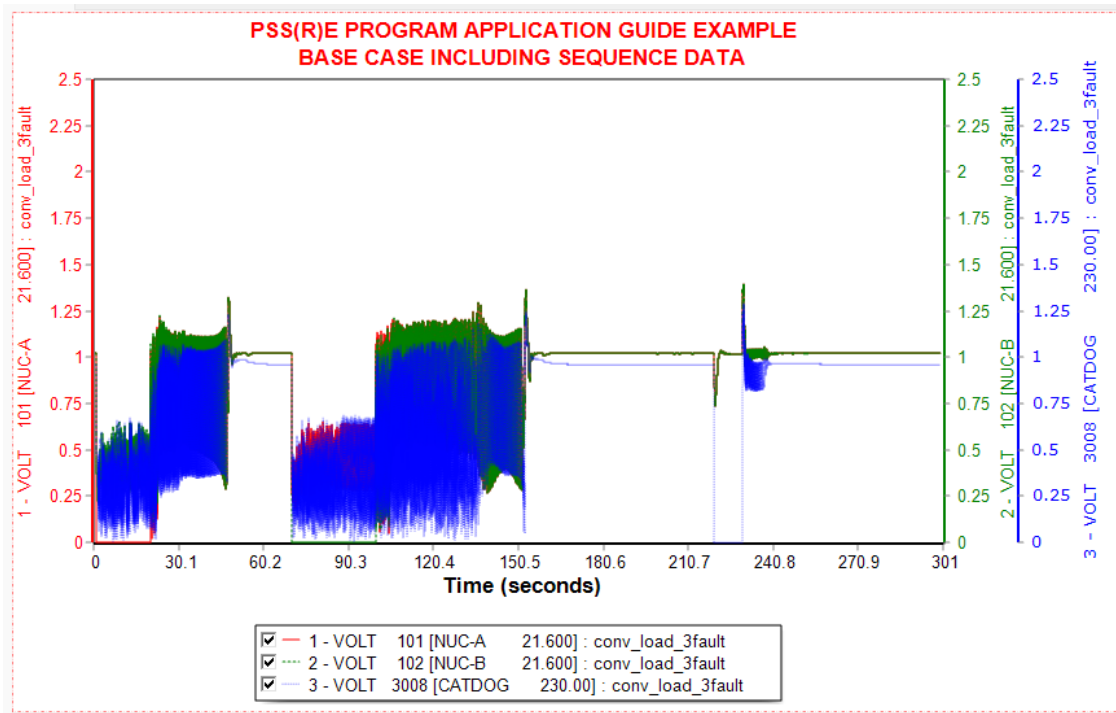


Figure 5: PSS/E Dynamic Results for Bus 101, 102, 3008 with Bus-to-ground Faults

which demonstrates that ePHASORSIM performs the same analysis as PSS/E, but does so in real-time, proving that there is a viable workflow from the case data transmission engineers currently use at VELCO to HIL simulation in RT-LAB.

While this workflow was identified, several challenges were encountered which ultimately prevented the team from successfully integrating VELCO's case with PSS/E. The primary challenge encountered was that the dynamic behavior of several critical devices on VELCO's system is governed by user-defined models, which are written to .dll files. These .dll files are not directly compatible with ePHASORSIM. The team's approach to work around this issue, which was ultimately not successful, is described below.

First, the team reduced the size of the case from all of New England down to Vermont territory only to reduce the number of user-defined models in the case. To do this, the team (with the help of a VELCO

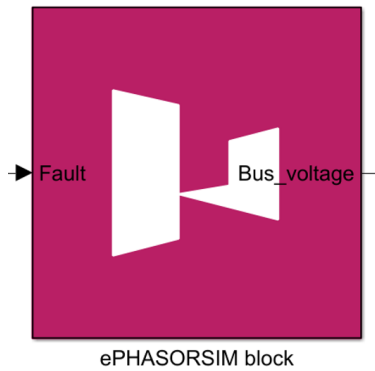


Figure 6: ePHASORSIM Block in RT-LAB

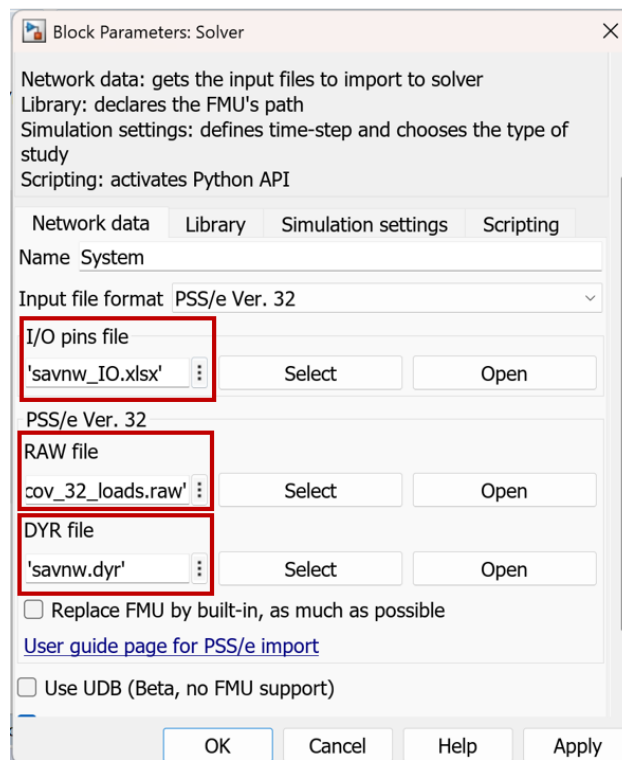


Figure 7: ePHASORSIM Parameter Popup in RT-LAB

	A	B	C	D	E
1	outgoing	Bus_voltage	bus_101/Vmag	bus_102/Vmag	bus_3008/Vmag
2	incoming	Fault	bus_101/active3PGFault	bus_102/active3PGFault	bus_3008/active3PGFault
3					
4					
5					
6					
7					

Figure 8: Excel Inputs/Outputs File

transmission planning engineer) equivalenced the case, which means that everything outside the desired Vermont territory was replaced with a load equivalent to the power consumption/injection at the boundary bus. Then, with the case equivalenced, the team attempted to 'GNET' any user-defined models that remained

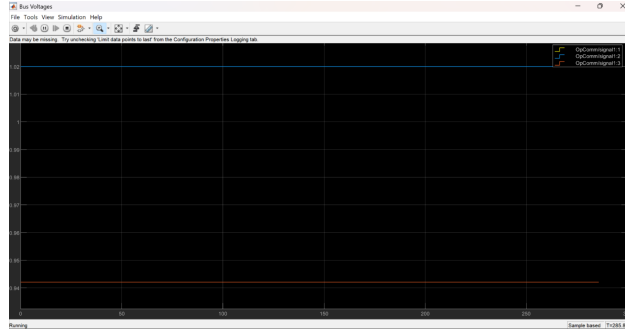


Figure 9: RT-LAB Dynamic Results for Bus 101, 102, 3008 with No fault

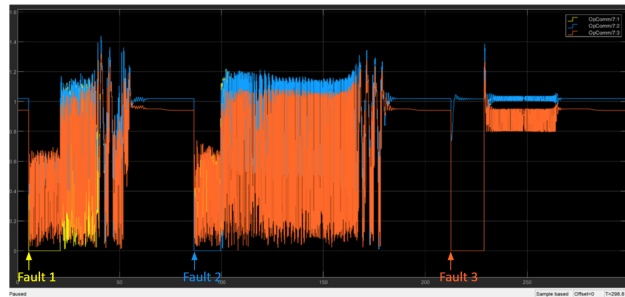


Figure 10: RT-LAB Dynamic Results for Bus 101, 102, 3008 with Bus-to-ground Faults

in the reduced territory. This means that the generators, which have complex dynamic behavior, were replaced with 'negative loads,' which have an equivalent real and reactive power consumption/production but have very simple dynamic behavior. This approach was ultimately ineffective because, with the simplified dynamic models, the case was no longer stable enough to converge, indicating an imbalance between load and generation. It was hypothesized that this failure was due to the nature of the 2033 Spring Daytime Minimum case, which has a high share of DERs – particularly solar generation – that are governed by user-defined models. Thus, substituting the complex dynamic behavior of these generators of the system with simple negative load models there was no longer enough dynamic flexibility to match load and demand on the system.

Therefore, an alternative approach was pursued. Rather than GNET-ing the user-defined models, the team attempted to replace the custom models with standard models that are available within ePHASORSIM, as shown in Figure 11, which is a table of available dynamic data models from the ePHASORSIM documentation page.

Synchronous machine	GENCLS, GENROE, GENROU, GENSAE and GENSAL.
Induction machine	CIMTR3
Excitation system	ESAC1A, ESAC5A, ESAC6A, ESDC1A, ESDC2A, ESST1A, ESST4B, EXAC1, EXAC2, EXAC4, EXDC2, EXPIC1, EXST1, IEEE11, IEEEEX1, IVOEX, SCRX, SEXS and URSTST.
Power system stabilizer	IEE2ST, IEEEET, PSS2A, PSS2B, STAB1, STAB3 and STAB4.
Turbine-governor	GAST, HYGOV, IEEEG1, IEEEG2, IEESGO and TGOV1.

Figure 11: Available Dynamic Models in ePHASORSIM

After replacing important dynamic devices in the case with alternative models, the case was still unstable and could not successfully run dynamic simulation. While unable to implement it during this project, the team did identify a path forward for VELCO to get their PSS/E case model to translate ePHASORSIM using Modelica, which will be explained further in the Review section.



Figure 12: The OP5033XG real-time simulator from OPAL-RT Technologies.

4.2.2 RT-LAB and OPAL-RT

For the project, the team was provided access to an OP5033XG real-time simulator, seen above in Figure 12 and referred to as the OPAL-RT throughout this report. The OPAL-RT is designed for ultra-fast, high-fidelity simulation of power systems, power electronics, and Hardware-in-the-Loop (HIL) applications. Built with multi-core Intel Xeon processors, large memory capacities, and support for analog and digital I/Os, the OP5033XG can support large-scale, detailed models operating at very small time steps, allowing for highly accurate real-time performance[3].



Figure 13: RT-LAB Software

To configure and run simulations on the OPAL-RT, the system is paired with RT-LAB, a software platform provided under the OPAL-RT license that allows for Simulink models to interact with the real world via the real time simulator. RT-LAB is accessed through a host PC and offers an interface that allows real-time simulation to be built, managed, and executed. RT-LAB supports the import of PSS/E files, such as .raw (version 32) and .dyr formats. This demonstrates the feasibility of integrating VELCO's PSS/E grid model for HIL with the OPAL-RT simulator. With the ePHASORSIM software package (explained above), PSS/E grid models can be translated into real-time simulation environments for dynamic and transient analysis.

The OP5033XG supports HIL testing due to its ability to both run in real time and interface with hardware. This allows physical devices, such as inverter controllers for this project, to be directly interfaced with the simulated grid environment. This makes it possible for hardware to be tested under a range of operating conditions including faults without endangering physical infrastructure.

The HIL capability of the OPAL-RT is valuable for this project, and more largely for transmission planning engineers at VELCO to study the effect of the growing presence of inverter-based distributed energy resources (DERs) in Vermont as explained in the problem definition. The OPAL-RT enables accurate modeling of power electronics, such as inverters, and their control strategies under real-time constraints. Ultimately, the OPAL-RT and RT-LAB environment form a robust platform for testing of Vermont's evolving power grid.

4.2.3 Inverter Controller

As discussed in the problem definition, inverters are an important connection for DERs on the grid. As the amount of DER increases so too will the number of inverters on the grid. These devices, especially when deployed at a large scale, can have massive impacts on the grid. To account for this, the HIL setup in Figure 2 uses an Arduino MEGA, coded as an inverter controller, as the hardware under test. As requested by the client, the inverter controller is coded using PQ control. In this case, PQ control regulates active (P) and reactive (Q) power output from the inverter modeled on Simulink. The Arduino MEGA, which is set up as shown in Figure 14, reads in direct voltage and current, quadrature voltage and current, and the real and reactive power reference values, as shown in a portion of the code in Figure 15. From there, the PQ control code creates a PWM based on the reference values it received, and outputs voltage waveforms used to generate the desired real and reactive power to the inverter and grid model.

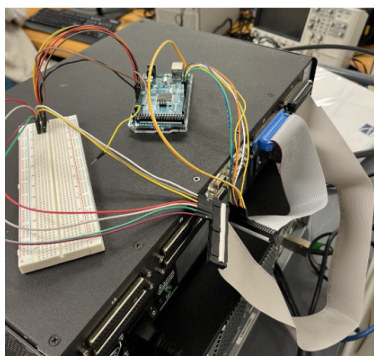


Figure 14: Arduino Mega setup with Opal-RT system

```
void loop() {
  // 1. Read Sensor Values
  float Vd = analogRead(A0) / 1023.0 * 5.0;
  float Vq = analogRead(A1) / 1023.0 * 5.0;
  float Id = analogRead(A2) / 1023.0 * 5.0;
  float Iq = analogRead(A3) / 1023.0 * 5.0;
  float P_ref = analogRead(A4) / 1023.0 * 5;
  float Q_ref = analogRead(A5) / 1023.0 * 5;
```

Figure 15: Portion of Arduino controller code

4.3 Alternative Design Considerations

4.3.1 Inverter to Arduino

Originally, the inverter shown in Figure 29 was going to be the hardware under test, but due to the safety restrictions of the Accelerated Testing Laboratory, the scope of the project shifted. By modeling the inverter on Simulink and coding the inverter controller on a microcontroller, no valuable concepts or analysis were lost for the client. Once the project scope shifted to using a microcontroller as the only hardware in the loop, the question became what microcontroller to use. First, the team tried to code a TMS320F28379D microcontroller from Texas Instruments, as this device came with the inverter and inverter model from the University of Texas. Once it was determined that the coding software required knowledge of C and a sharp learning curve in little time, the team decided using an Arduino UNO would be more effective. The code for the Arduino UNO was on its way to success, until the number of pins needed exceeded that offered on the UNO. The Arduino MEGA combatted this issue with its larger board and pin count. With this, the PQ controller was successfully built and able to interface with the Simulink model and OPAL-RT system.

5 Design Evaluation

5.1 Technical Analysis

5.1.1 Modeling the Grid in RT-LAB

The digital component of the HIL simulation was run in RT-LAB. Within RT-LAB, both the grid and the inverter were modeled using Simulink blocks, including from the SimScape library. (The inverter controller was the only piece of physical hardware integrated into the HIL, connected through the OPAL-RT.) It was up to the team to decide how to model the grid in this simulation.

To start, the transmission system was modeled using ePHASORSIM by importing a .raw and .dyr file containing the steady-state data and the dynamic data for the case, respectively. Analysis was not required to make this decision because ePHASORSIM was provided by the vendor as a tool for running dynamic simulation in OPAL-RT. However, ePHASORSIM only models what is contained in the case files, which ends at the transmission system (in this case, at a voltage of 21.6 kV). Meanwhile, inverters are connected on the distribution system, which goes down to a voltage of 120 V for consumer use. Additionally, ePHASORSIM runs *single-phase* dynamic stability analysis, which is a simplification of the three-phase transmission system. This means that it only outputs one per-unit voltage signal to represent three sinusoidal voltage signals at any given bus in the network (the equation for per-unit voltage is given in 1). Therefore, the team had to construct a model of the grid that transformed the single-phase output of ePHASORSIM into the necessary three-phase, low-voltage signal to connect to the inverter.

$$V_{pu} = \frac{V_{actual}}{V_{base}} \quad (1)$$

Figure 16 shows the grid model that the team designed. As shown in the figure, the model begins with ePHASORSIM, where the case data from PSS/E is imported and the dynamic simulation occurs. The output of the ePHASORSIM solver block is the single-phase, 21.6 kV base per-unit voltage, PU_V. (In the diagram, the voltage at three busses is monitored, but only one bus is connected to the inverter, which is why the de-mux is included.) This per-unit voltage signal is then passed to three of the blocks shown again in Figure 17.

Figure 29 shows the per-unit voltage scaled by the base voltage, T, and transformed into a sinusoid. The sinusoidal blocks also each contain an angle offset to make the sinusoids 120 degrees apart. This transformation is summarized by Equations 2 - 4.

$$va(t) = T \sin(\omega t) \quad (2)$$

$$vb(t) = T \sin(\omega t - \frac{2\pi}{3}) \quad (3)$$

$$vc(t) = T \sin(\omega t + \frac{2\pi}{3}) \quad (4)$$

These three-phase simulink signals are then passed into SimScape controlled AC voltage source blocks, which convert them to voltage signals. Finally, the transformer (highlighted in Figure 18) steps the voltage down to the 120 V level.

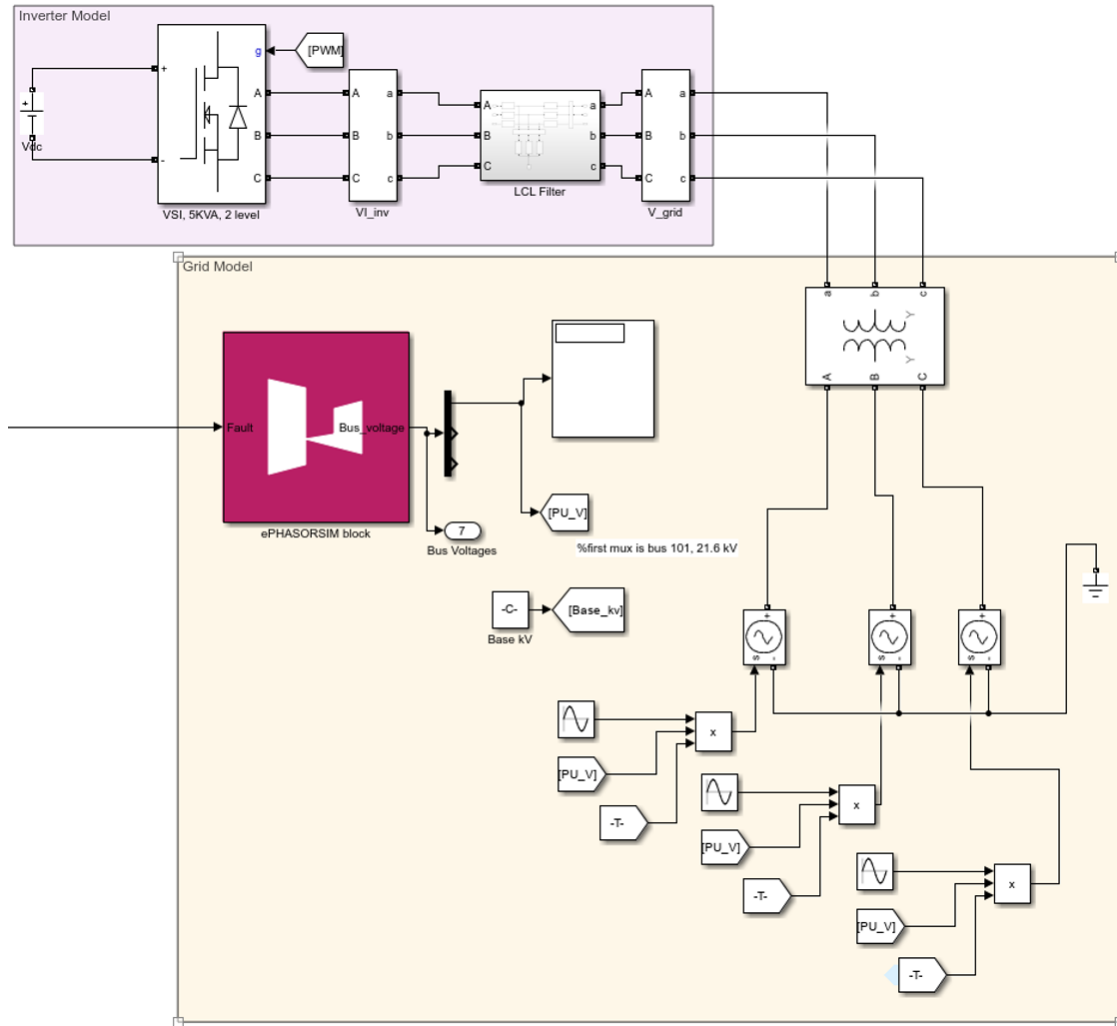


Figure 16: Grid Model Connected to Inverter in RT-LAB

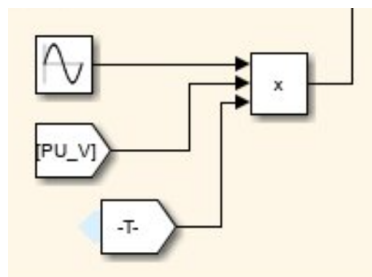


Figure 17: Per-Unit to AC Voltage in RT-LAB

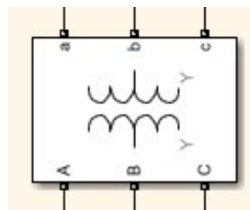


Figure 18: SimScape Three-phase Transformer in RT-LAB

5.1.2 Choosing a Step Size

A key objective of the project was to simulate in real-time. The reason why simulating in real-time is so important to the project is because the HIL simulation aims to collect the actual response of a physical device to a simulated condition. Unlike a fully digital simulation, the hardware would behave differently if the same condition were simulated at a slower or faster rate than real-time.

To choose an appropriate step size, the team performed a trial-and-error analysis with several different time steps. The first time step size tested was 10^{-3} seconds. The simulation setup page is shown in Figure 19 which depicts the settings configuration for a step size of $1e-3$. Figure 20 displays the error message that is generated when HIL was attempted with a time step of 10^{-3} seconds, which was ultimately too large. The error message explains that the simulation was stopped likely due to numerical instability due to insufficient time resolution, which was the case. The solver tried to compute the rate of change of a variable (its derivative) between time t and $t+t$. But due to the large timestep (t), it overstepped a steep slope or discontinuity, and the computed derivative became Infinte or NaN-not a finite number-as expressed in the error message. The time step for the simulation needs to be sufficiently small so that transients can be captured.

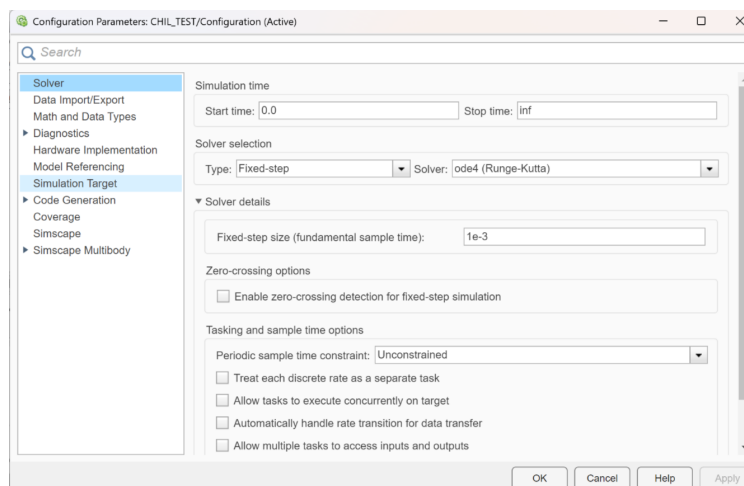


Figure 19: Setting Step Size to $1e-3$ in RT-LAB

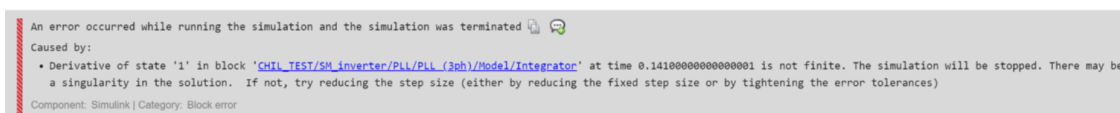


Figure 20: Error Message for $1e-3$

The next time step size trialed was 10^{-6} seconds. The simulation was able to run for this timestep, but it could not solve fast enough to maintain real-time operation. The filter voltage output from this simulation can be seen in Figure 21. While the 3 phase voltage waveforms look accurate, the simulation was unable to solve quick enough to operate in real time as indicated by the simulation timed falling out of sync with wall-clock time. This confirms that a time step of 10^{-6} seconds, although numerically stable, was too small to meet real-time constraints on the available hardware.

With the knowledge that a time step of 10^{-6} seconds was too small and 10^{-3} seconds was too large, the team next tested out 10^{-4} seconds. This timestep allowed the simulation to run and also maintain real time. As determined from the trial and error analysis, finding the correct timestep is critical to preform a real-time HIL simulation.

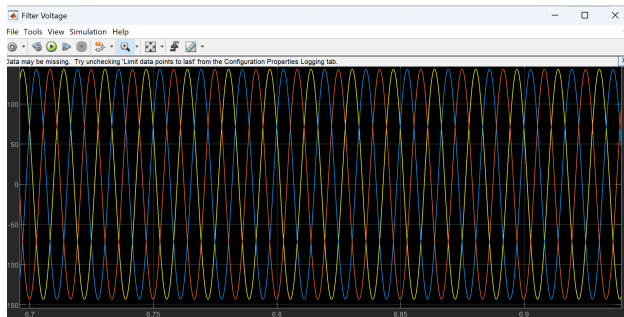


Figure 21: Filter Voltage Output

5.1.3 Using an Arduino Mega Microcontroller

To decide which microcontroller to use to implement the inverter controls, the team had to analyze the specifications of a few different options. These options were: a TMS320F28379D, Arduino UNO, and Arduino MEGA. First, the team tried to code a Texas Instruments TMS320F28379D microcontroller, since this device met all project specifications. The TMS320F28379D came with the inverter and inverter model from the University of Texas, which meant it would have been a good option to further the project in the future. Once it was determined that the coding software required knowledge of C and a sharp learning curve in little time, the team decided using an Arduino UNO would be more effective. The code for the Arduino UNO was on its way to success, until the number of pins needed exceeded that offered on the UNO. The Arduino UNO has 6 PWM output pins, 6 analog input pins, and 6 power pins. The Arduino MEGA combatted this issue with 15 PWM output pins, 16 analog input pins, and 6 power pins. With this, the PQ controller was successfully built and able to interface with the Simulink model and OPAL-RT system.

5.1.4 Scaling Signals for Arduino

The inverter controls were programmed onto an Arduino Mega. The inverter controller takes several values as inputs. The first are V_d and V_q , which are the direct and quadrature voltages, respectively (these come from the DQ transformation of the three-phase grid voltage). Similarly, I_d and I_q are the direct and quadrature currents. P_{ref} is the active power reference signal, which ‘tells’ the inverter controller what active power to track (i.e. how much power to send from the PV panels to the grid). $Freq$ is the frequency of the grid voltage, which is critical for matching the inverted DC power from the panels to the AC power flowing on the transmission system. Finally, θ is the angle used for the DQ transform, which allows V_d , V_q , I_d , and I_q to be mapped back to three phases after they have been used in the control loop.

Each of these signals had to be passed from the simulated model of the grid and the inverter in RT Lab to the inverter controls on the Arduino. The Arduino, meanwhile, can only read values within a range of 0-5V. Thus, the team had to perform an analysis to determine how to appropriately scale the voltages from the RT-LAB model to the 0-5V range of the Arduino.

The first step in the analysis was to run an offline simulation (meaning, the simulation was run using Simulink and not at real-time pace) and to use scopes in the Simulink model to view V_d , V_q , I_d , and I_q to get an expected range for each value. Figures 22 and 23 show the results of this step.

For the remaining inputs, P_{ref} , $freq$, and θ , the range was already known; P_{ref} is specified by the simulation operator in the RT-LAB model, $freq$ is based on the grid frequency (which is 60 Hz), and θ ranges from 0-10. Table 2 gives a summary of the ranges selected for each value based on their known or simulated minimum and maximum.

To implement these results in the design once the ranges were identified, the values were scaled down to the 0-5V range in RT-LAB, then reverse-engineered back to their original values on the Arduino. Figure 24 shows the scaling in RT-LAB and Figure 25 shows the code on the Arduino for recovering the signals. In Figure 24, the offset values ‘ V_d offset,’ ‘ V_q offset,’ and ‘ I_d offset’ are defined and added to their respective signals to shift the signal upward and make it fully positive. Then, a gain block is applied to each signal which divides it by double the offset value, scaling it down to 0-1, then multiplies by 5, scaling to the

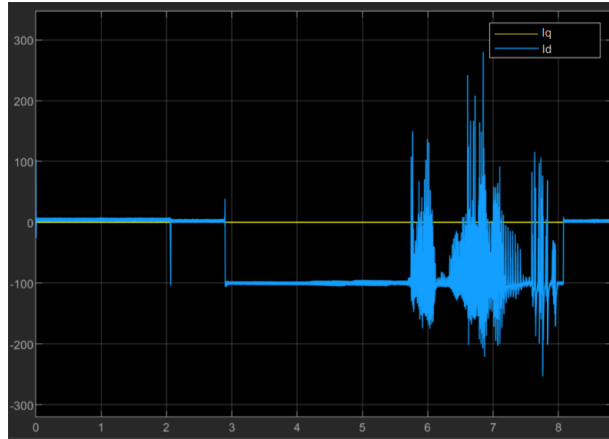


Figure 22: Current Signals from Offline Simulation

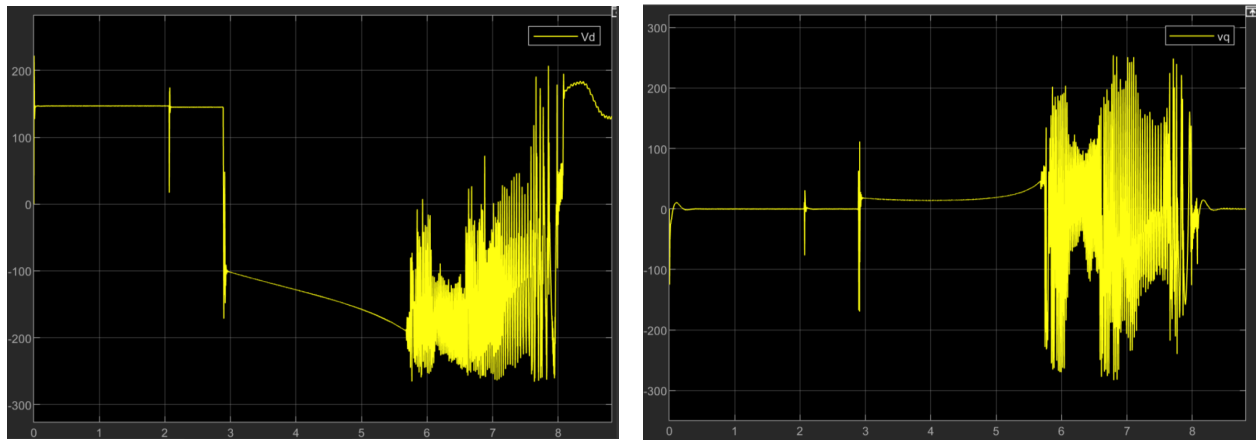


Figure 23: Voltages Signals from Offline Simulation

Arduino-appropriate range of 0-5 V. In Figure 25, the inverse process is performed for each signal. The `analogRead` function in Arduino, however, returns a value between 0 and 1023 corresponding to the 0-5V voltage it reads at the pin. Thus, before the inverse process is applied, the value must be mapped from 0-1023 to 0-5 by dividing the value by 1023 and multiplying by 5.

Signal	Min	Max
Vd	-275	275
Vq	-300	300
Id	-300	300
Iq	0	1
Pref	0	500
Freq	0	100
Theta	0	10

Table 2: Controller Input Signal Ranges

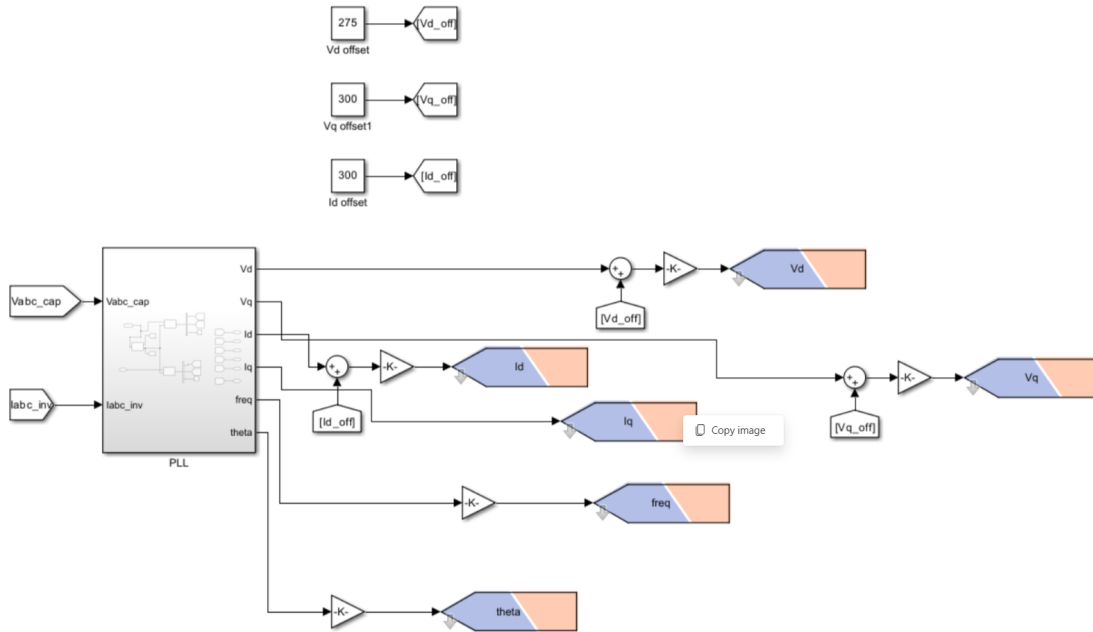


Figure 24: Scaling Signals in RT-LAB

```

188 // Scaling offsets based off of expected value range
189 // Offsets determined from Software-in-the-loop simulation
190 float Vd_offset = 275;
191 float Vq_offset = 300;
192 float Id_offset = 300;
193
194 // Reading in and scaling values from the OPAL-RT
195 float Vd = (analogRead(Vd_pin) / 1023.0) * 5.0 * ((2 * Vd_offset) / 5.0) - Vd_offset;
196 float Vq = (analogRead(Vq_pin) / 1023.0) * 5.0 * ((2 * Vq_offset) / 5.0) - Vq_offset;
197 float Id = (analogRead(Id_pin) / 1023.0) * 5.0 * ((2 * Id_offset) / 5.0) - Id_offset;
198 float Iq = (analogRead(Iq_pin) / 1023.0) * 5.0;
199 float P_ref = (analogRead(P_ref_pin) / 1023.0) * 5.0 * (500 / 5.0);
200 float freq = (analogRead(freq_pin) / 1023.0) * 5.0 * (100 / 5.0);
201 float theta_read = (analogRead(theta_pin) / 1023.0) * 5.0 * (10 / 5.0);

```

Figure 25: Arduino Code for Scaling Signals

5.2 Engineering Specification Evaluation

Engineering Specifications		Results		
Engineering Specification	Units	Verification Method	Verification Result	Verification Status (Pass/Fail/Not Verified)
Simulation shall have a step size of less than 10 milliseconds.	ms	Demonstration	The fixed model step size was set to 100 microseconds which is less than 10 milliseconds.	Pass
Simulation shall collect inverter model response for at least 3 seconds.	s	Demonstration	Simulation collects inverter model response for 9 seconds, which is greater than 3.	Pass
Simulation shall plot voltage values for at least 3 points in the system.	s	Demonstration	The simulation successfully plots voltages for at least three points in the system.	Pass
CHIL simulation shall use less than 10 Wh per run.	Wh	Test	The power meter read 0.00 Wh, meaning less than 10 Wh were consumed.	Pass
CHIL system shall use an Arduino Uno microcontroller and an Opal-RT.	None	Inspection	The simulation used the OPAL-RT simulator and Arduino. An oscilloscope was used to confirm that a voltage signal was sent to the arduino from the OPAL-RT, and that a signal was being sent from the Arduino to the OPAL-RT.	Pass
Simulation shall solve the system within 1 μ s less than the simulation step size.	μ s	Demonstration	The voltage values from the real-time simulation are as expected, indicating that the real-time simulator successfully solved within 1 microsecond of the specified 100 microsecond timestep, which is less than the 10 milliseconds specified.	Pass
Simulation shall use less than 1 GB of RAM.	GB	Demonstration	The simulation used 3.55 GB of memory, which is bigger than 1	Fail
Instructions for replication shall be reviewed and approved by 3 transmission planning engineers at VELCO.	Engineers	Survey	The delivery date for the documentation package has yet to come, and therefore the package has not been reviewed by any VELCO engineers yet.	Not Verified
Simulation shall be run for 3 different fault conditions.	Conditions	Demonstration	The simulation successfully collected bus voltages at three busses for three fault conditions	Pass
Simulation shall model grid conditions for network size of at least 800 busses.	Busses	Demonstration	The HIL simulation ran for a 27 bus system which is less than 800 busses.	Fail

Figure 26: Engineering Specifications

The project had a relative pass rate of 73%. Two specifications failed, and one has yet to be verified. The first specification that failed was 'Simulation shall use less than 1 GB of RAM'. This specification was set by the SEED team out of interest to ensure that the simulation was compatible with any system and overall lightweight. After verifying this specification through demonstration, it was determined that the simulation used 3.55 GB of RAM. While this specification ultimately failed, the team decided that this was not a major drawback of the project. The team had limited ability to minimize the memory demand of the simulation since there were restrictions on the software that needed to be used. The team also determined that a memory size of 3.55 GB is sufficiently small to run on any PC that would be used for HIL simulation. Most PCs have at least 8 GB, if not 16 GB or more of available RAM.

The second specification that the project failed to meet was: 'the simulation shall model grid conditions for a network size of at least 800 busses'. The project successfully ran an HIL simulation for a 23-bus system, but was unable to run HIL with VELCO's network with over 800 busses. The team verified that ePHASORSIM is capable of modeling a network of 800 busses; in fact, it can model a network of over 10,000 busses according to OPAL-RT's documentation. The inability to run HIL with VELCO's network modeled in PSS/E was instead due to file incompatibilities between softwares. As explained in the system overview, the dynamic behavior of specific equipment in a PSS/E case is defined through .dll files. To run the HIL simulation, PSS/E data is loaded into the ePHASORSIM solver block, which does not accept .dll files directly. Without the ability to load the information from the .dll files into ePHASORSIM, dynamic stability analysis was unable to run in HIL with the VELCO network. To get around this limitation, the team worked to modify the PSS/E case to be compatible with ePHASORSIM, but was ultimately unsuccessful because after GNET-ing the generators on the scaled VELCO case or replacing them with approximate generic models, the network was no longer able to converge.

Working with the client and the vendor, the team eventually identified a viable path forward to allow the VELCO network to be compatible with ePHASORSIM while accurately modeling the dynamic behavior of all machines on the network. This can be achieved using Functional Mockup Units (FMUs), which allow for co-

simulation and information exchange between different simulation platforms. An FMU is essentially a zipped file that contains a system description file (.xml), a binary file (.dll), and any additional model files such as .csv or .xlsx. FMUs can be generated from Modelica models, and Opal-RT provides documentation and tools to support this process. Specifically, their FMUcreator tool enables users to convert OpenModelica or Dymola models into FMUs that are compatible with ePHASORSIM, streamlining integration and ensuring accurate dynamic modeling of system components. Once the zipped file has been created, it can be incorporated into the ePHASORSIM dynamic simulation as shown in Figure 27, where the path specified in FMU component corresponds to a folder containing all FMUs necessary for the simulation.

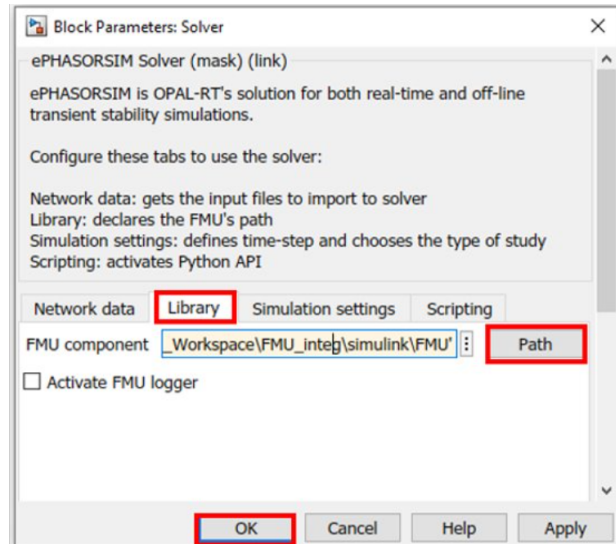


Figure 27: Inclusion of FMUs in ePHASORSIM Solver

While the team was unable to pursue this path with the balance of time left on the project, the client has indicated that obtaining models of the dynamic behavior of their custom devices in Modelica would be feasible through their Common Information Model (CIM), making this path forward viable for the future of the project.

The final specification that was not verified was: 'instructions for replication shall be reviewed and approved by 3 transmission planning engineers at VELCO.' While this specification was not met by the project submission date, the team has been in frequent communication with the client to ensure that a comprehensive documentation package will be prepared containing all desired information. The team will deliver this package to VELCO by May 2nd, 2025.

5.3 Budget Analysis

VELCO provided a project budget of \$1,500.00. Throughout the project duration, the team spent \$0.00. The project relied on the use of expensive software, including PSS/e, Simulink, MATLAB, and ePHASORSIM, and very expensive hardware, the OPAL-RT system. These tools were provided to the team by UVM.

The team determined that HIL simulation is possible with the tools currently available at UVM. If there were the need for additional software, it would have exceed the project's budget. The fact that HIL was accomplished with the tools available to VELCO through UVM is a vital finding of the project.

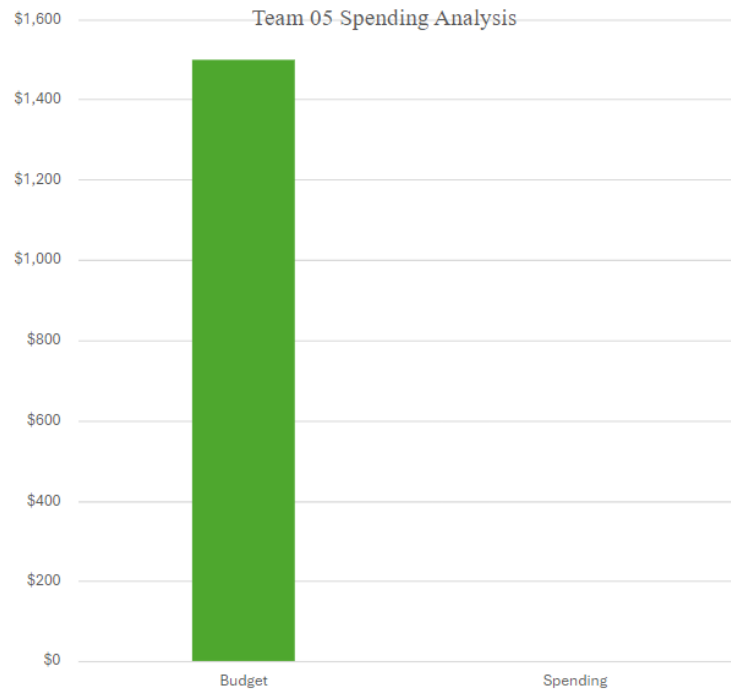


Figure 28: Team 5 Spending

6 Project Reflections

6.1 Retrospective

Throughout the project's duration, both the efficiency of the project and the team's dynamic have improved. The most impactful change the team implemented throughout the project to speed up progress was the delineation of tasks. Initially, the team attempted to work through every step of the project together, which led to slow progress and inefficient use of resources. Although this approach helped everyone start on the same page, it quickly became clear that a change in process was necessary to improve efficiency. The team first split in to two groups of two with clear overarching tasks. Within each pair, specific smaller tasks were also defined. This explicitness on task break down allowed for each team member to have clear instruction on what they needed to accomplish. While even at the end of the project the team was still working to improve on this, this change made early on in the project had cascading effects on the efficiency of the project. At times when tasks were left unclaimed, progress was slowest. An example of the effect of this change could be seen from the different weeks of each sprint. At the start of the sprint the team would identify many tasks that needed to be completed, but not explicitly assign them to a individual team memeber yet. Then, the first week of the sprint would pass and little progress was made on the large to do list. Then on the second week, the team realized all that had to be done within the next two weeks and start claiming and assigning tasks. Once this was done, the rate of progress made by the team accelerated.

Sprint reviews were a valuable aspect of the project, providing structure and accountability. They created deadlines that helped ensure consistent progress, offered opportunities to reflect even when momentum felt slow, and kept the team's actions aligned with the client's expectations. For a long-term project, these regular check-ins helped the team stay on track and meet necessary milestones.

In the middle of the project, when the team encountered repeated setbacks, the sprint reviews were especially helpful. They provided a time to look back at the work accomplished, even when tangible progress felt limited. They also allowed the team to communicate roadblocks directly with the client. Hearing that the identification of issues was still valuable to the client was validating and motivating.

Each sprint review also gave the client a chance to redirect the team when needed, helping ensure that priorities stayed aligned. At times, this meant refining the project scope to focus on the most critical elements. For example, when the team faced accessibility issues with the new ATL lab and could not use the physical inverter, the client supported a pivot toward using a microcontroller as the hardware for hardware-in-the-loop (HIL) testing—preserving the core goal of the project. Overall, the sprint reviews were an effective tool for maintaining alignment between the team and the client.

The team did not utilize the scrum board to its full potential to manage tasks. The scrum board was mainly consulted on the first and final day of each sprint. If the scrum board had been used more frequently, the team would have had a structured way to delineate tasks.

6.2 Review

As the project concludes, the team identified 3 open issues along with recommended actions to be taken. The first and most important issue is that user-defined models in VELCO case cannot be imported directly to ePHASORSIM in the form of DLL files. The team's recommendation to solve this issue is to use open Modelica to generate FMUs for user-defined models in VELCO case.

Another open issue is that in the project's current state, only the transmission system is modeled in ePHASORSIM while the distribution system is approximated. The distribution system is where the majority of the inverters of interest exist. By co-simulating both the transmission and distribution system, the simulation would be more robust and should yield more accurate results.

The last open issue is that the team did not interface HIL with the physical inverter. The recommended action for this issue is to get the proper training to access the ATL lab once it is finally set up. This may not be possible for undergraduate students due to the dangerous nature of a high voltage lab. If this were the case, the team could instead interface with a scaled down inverter that works with safer voltage levels.

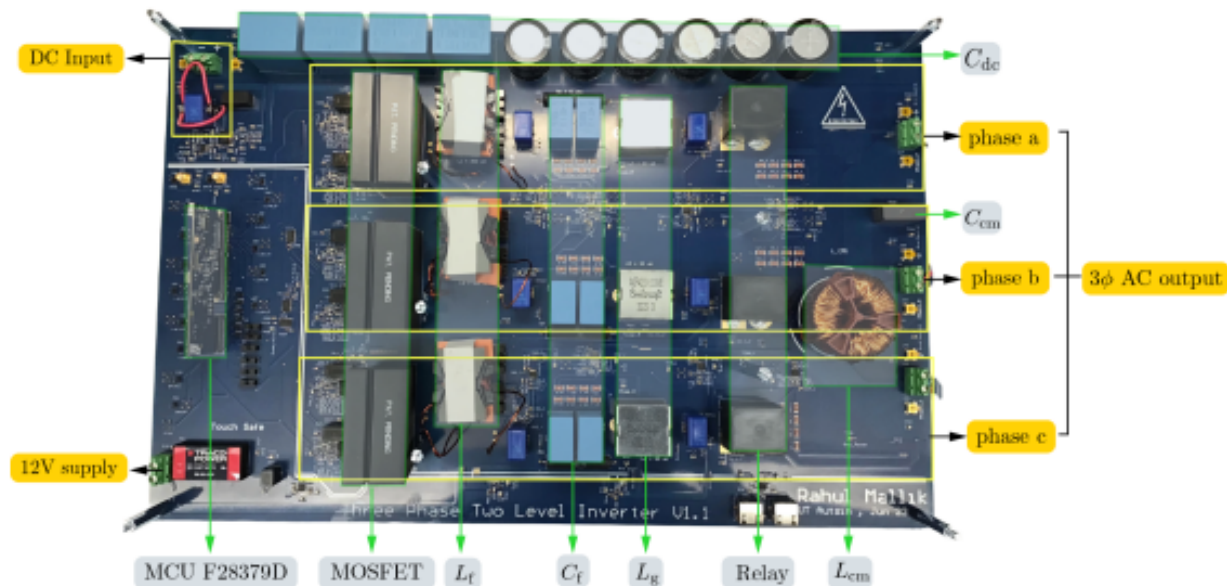


Figure 29: Physical Inverter PCB Layout

At the final design presentation and Design Night—as well as throughout the duration of the project—the client expressed a high level of satisfaction with the team's work. From the outset, the client made it clear that they did not have rigid expectations for a specific project outcome. Instead, they emphasized the importance of the team gaining meaningful learning experience.

Unlike many SEED projects, which typically result in a tangible final deliverable, this project focused on exploring the feasibility of conducting hardware-in-the-loop (HIL) simulation using the tools available to

VELCO through their partnership with UVM as a way to evaluate inverter responses to grid fault scenarios. The deliverable for this project is a comprehensive documentation package detailing what approaches were unsuccessful, what approaches were successful, and how HIL could be implemented with the given tools. The client was satisfied that the team successfully ran a HIL simulation using an Arduino microcontroller.

6.3 Lessons Learned

Throughout the project, the team gained valuable experience on integrating multiple software platforms while deepening their understanding of power systems and the importance of transmission planning. The team learned how to navigate expectations between the client, team members, and course instructors. The team also got experience with managing a year long project including the alignment of short term and long term deadlines.

One of the most significant lessons learned was the importance of upfront planning. At the outset, the team experienced some discrepancies regarding project expectations from both the client and mentors. In hindsight, an initial meeting with the client, team, and mentor to clearly outline the project's goals and success criteria would have been highly beneficial. Although fulfilling all requests added value, a clearer alignment might have ensured that early work directly supported later efforts, particularly the implementation of hardware-in-the-loop in the second semester.

The team recognizes that this situation could have been better managed by proactively seeking clarification and asking questions when the connection between tasks and project objectives was unclear. Towards the end of the first semester, the team had a clarifying conversation with the client on project expectations which the project may have benefitted from had the conversation occurred earlier. This also ties into another key lesson learned: bridging communication gaps between groups with different knowledge backgrounds is essential for project success. The client and mentors entered the project with significantly more background knowledge, which likely contributed to differing interpretations of project goals and priorities among the team, client, and mentor. Recognizing and addressing these differences early on could have helped align perspectives and improve collaboration throughout the project.

The team will benefit in the future from having learned the importance and impact of having clear expectations at the start of a project. While project paths are often not linear or fully defined, identifying key priorities early on is critical to ensuring satisfaction between a team and the client.

Another key lesson learned during the project was the importance of adaptability. Sometimes a pivot in project scope is necessary to yield a successful product. As mentioned earlier in the report, in the month of March the team had to change a major aspect of the project - from using a physical inverter to using a microcontroller for hardware. The team had been waiting for UVM's new ATL lab to be completed to start working with a physical inverter due to its high voltage nature. Unfortunately, the lab was finished a few months after expected and also the team found out that undergraduate students were not allowed to be working in that lab without the guidance of a trained graduate student.

With this information, the team knew it was necessary to pivot to still be able to interface the OPAL-RT with a piece of hardware to prove whether HIL was feasible with the tools available at UVM, the main question this project was addressing. After doing some research and consulting with project advisors and the client, the team chose to use a microcontroller to represent the inverter controller. This choice met safety requirements, fulfilled the majority of client expectations, and provided a significant step toward future HIL work with a full inverter. Since the inverter controller acts as the "brains" of any inverter system, implementing it was still highly relevant and impactful.

This decision allowed the team to successfully run a HIL simulation within the project timeline. Had the team insisted on working with the physical inverter, key project goals might have gone unmet in the pursuit of a "perfect" deliverable.

Although there was initial disappointment about not using the physical inverter, the ability to pivot and realign with the project's core objective was critical to its success. The team learned that delivering the best possible solution means keeping the client's needs at the forefront, even when it means letting go of the original plan.

References

- [1] State of Vermont Department of Public Service, “2022 vermont comprehensive energy plan.” https://publicservice.vermont.gov/sites/dps/files/documents/2022VermontComprehensiveEnergyPlan_0.pdf, 2022. Accessed: Dec. 8, 2024.
- [2] ISO New England, “Evolving light load, high voltage transmission system problems and distribution system interaction.” https://www.velco.com/sites/default/files/2023-12/T-D-technical-forum-mtg1_FINAL.cleaned.pdf, 2023. Accessed: Dec. 8, 2024.
- [3] OPAL-RT Technologies, “Op5033xg real-time simulator datasheet.” https://www.opal-rt.com/resource-center/document/?resource=Mkt_0027596. Accessed: Apr. 29, 2025.